SDR-based Design of Spectrum Sensing Systems for monitoring of FM broadcasting

Key Properties of SDR-Based Monitoring under Design

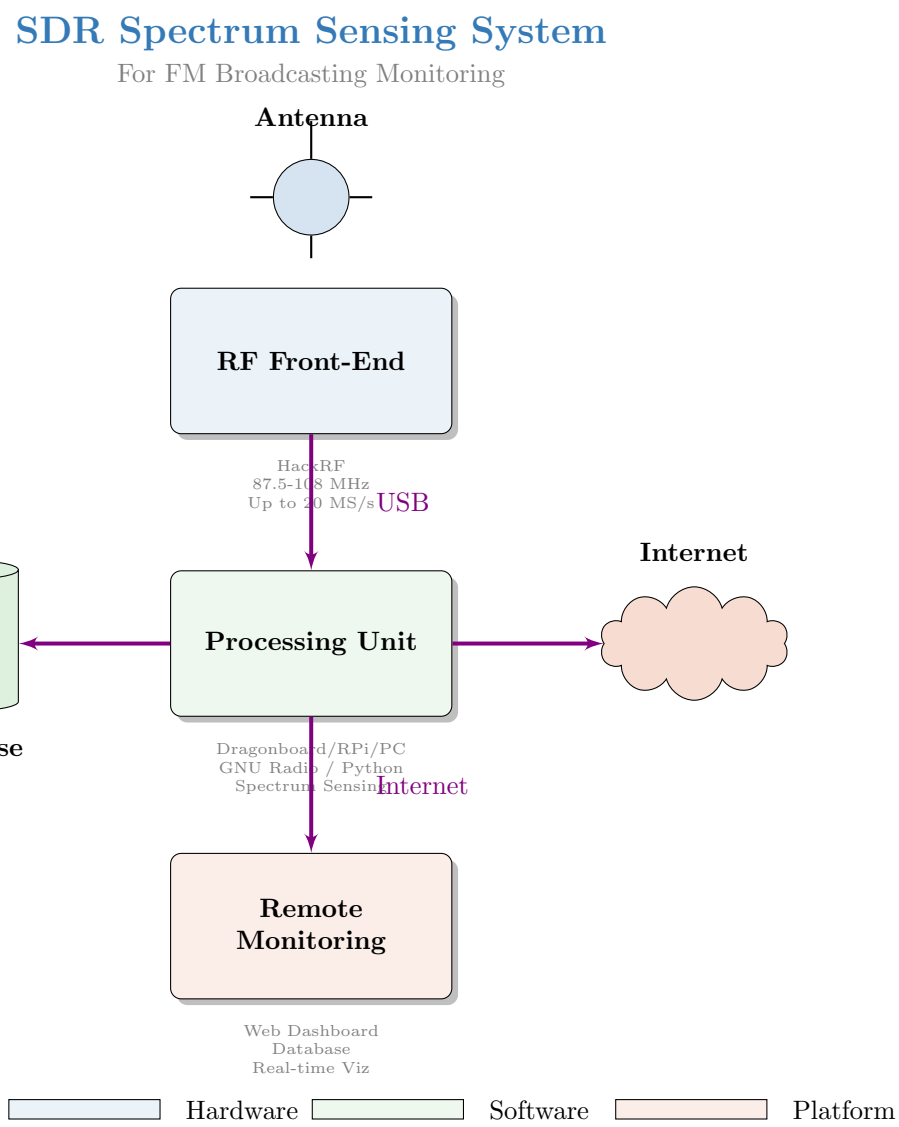
- **Reconfigurability and Flexibility:** The SDR platform must have reconfigurable ability and flexibility to set communication parameters when monitoring radio signals. Because its core functions demodulation, detection thresholding, channel occupancy analysis, and compliance reporting are implemented in software rather than dedicated circuitry, the system must be capable of being adapted to accommodate new regulatory requirements, extended to monitor adjacent frequency bands, or enhanced with advanced detection techniques such as cyclostationary feature analysis or machine learning-based signal classification, all through remote software updates and without hardware replacement.

- **Cost-Effectiveness:** The HackRF One platform combined with embedded processing systems (Dragonboard 410C, Raspberry Pi) must enable permanent spectrum monitoring at a fraction of the cost of traditional dedicated spectrum analysers, which typically command acquisition costs in the range of $10,000–$50,000 USD per monitoring channel. The total deployed cost per sensing node including SDR hardware, embedded processor, antenna, LNA, enclosure, and installation must not exceed $500 USD, representing an order-of-magnitude cost reduction that enables large-scale distributed deployment across a regulatory monitoring network.

- **Remote Monitoring Capability:** The SDR-based system must transmit captured broadcast content and compliance measurement data to a centralised server via the internet, enabling regulatory staff to monitor multiple FM stations remotely without visiting distant studio or transmitter sites. All data transmission must be secured using encrypted communication channels (TLS 1.3 minimum or equivalent VPN tunnel), and access to the remote web dashboard must be restricted to authenticated and authorised regulatory personnel through a role-based access control mechanism.

- **Multi-Station Monitoring:** The software processing pipeline must support the simultaneous real-time demodulation and compliance monitoring of no fewer than two FM broadcasting stations on the target embedded platform, with the architecture designed to scale to additional channels as processing headroom permits. Monitoring throughput shall be verified through benchmarking on the target hardware prior to operational deployment, and the maximum sustainable number of simultaneously monitored channels shall be documented as part of the system performance specification.

Core Architectural Components



RF Front-End (Hardware Layer): At the physical layer, a low-cost SDR receiver – HackRF One – serves as the antenna-coupled front end. Its role includes: receiving electromagnetic energy from the environment, down-converting it to an intermediate or baseband frequency, and digitizing it via an analog-to-digital converter (ADC). The critical parameters at this stage are the receiver's frequency range, dynamic range, noise figure, and sampling rate, all of which determine the fidelity with which the FM band can be captured. A wideband antenna appropriately tuned to the VHF-II band feeds the SDR, and a low-noise amplifier (LNA) may be inserted in the signal chain to improve sensitivity and compensate for feedline losses.

Signal Acquisition and Digitization: The SDR hardware streams raw I/Q (in-phase and quadrature) samples to a host computer via USB, or Ethernet. These complex baseband samples represent the entire instantaneous bandwidth captured by the receiver as part of the FM broadcast band in a single sweep, depending on the SDR's maximum sample rate. This wideband capture capability is a decisive advantage: rather than tuning sequentially to individual channels, the system can simultaneously observe all FM channels within its instantaneous bandwidth.

Data Management and Reporting Layer: Processed results feed into a database backend structured around time-stamped measurement records enabling historical trending, statistical compliance reports, and event-based alerting. A web-based dashboard or API layer allows regulatory staff to query the system remotely, visualize spectrograms and occupancy heatmaps, and generate audit-ready compliance documentation. Automated alert mechanisms (email, SMS, or API webhooks) can notify engineers when threshold violations are detected.

Software Processing Engine: The processing engine implements the end-to-end digital pipeline for *FM broadcast compliance monitoring*. Rather than enabling spectrum access, it produces *auditable measurements, evidence records, and compliance reports*. Its layered tasks include:

1. **Wideband Spectrum Sensing (Occupancy & Discovery).** Estimate the wideband PSD, detect active carriers, and maintain a time-stamped occupancy map of the FM band. Key outputs include detected station candidates (center frequency estimates), received power proxies, and activity timelines.
2. **Channelization & Station Tracking.** Isolate each detected station using channelization (e.g., FFT/PFB channelizer) and track (i) center frequency drift, (ii) signal level trends, and (iii) continuity/outage events over time.
3. **Signal Identification (Service/Station Classification).** Classify emissions in an FM-monitoring sense: confirm FM broadcast characteristics (e.g., deviation profile), detect stereo pilot (19 kHz), and optionally decode RDS for station ID metadata. This step supports *station attribution* and reduces false alarms from non-broadcast interferers.
4. **Compliance Feature Extraction.** Compute regulatory-relevant metrics per station, such as: occupied bandwidth, adjacent-channel leakage indicators, spurious/abnormal spectral components, and carrier frequency error/drift. These features feed rule checks and reporting.
5. **Compliance Assessment & Alerting.** Apply configurable decision rules (thresholds, spectral masks, persistence criteria) to flag potential non-compliance events, generate alerts, and annotate events with confidence scores.
6. **Evidence Packaging & Reporting.** Store time-stamped measurement summaries and optional short evidence snippets (I/Q or audio) with integrity controls (e.g., hashing/signing), and generate periodic compliance reports (daily/weekly) plus event-driven incident logs.

Wideband Spectrum Sensing (Occupancy & Discovery).

This task estimates the distribution of received power across frequency to (i) discover active FM emissions, (ii) maintain an occupancy map of the band over time, and (iii) provide measurement features that feed station tracking and compliance checks.

1. **Spectral Parameter Estimation.** Given a PSD estimate $\widehat{S}_x(f)$ over a band of interest, derive the following band-level features:

   - *Power at specific frequencies:* $\widehat{S}_x(f_k)$ for selected f_k (e.g., nominal channel centers).
   - *Peak frequency:* $f_{\text{pk}} \triangleq \arg\max_f \widehat{S}_x(f)$ (candidate carrier location).
   - *Occupied bandwidth:* B_{occ} defined as the smallest bandwidth around a center f_c that contains a prescribed fraction of in-band power (e.g., 99%), or equivalently via a thresholded support set (see below).

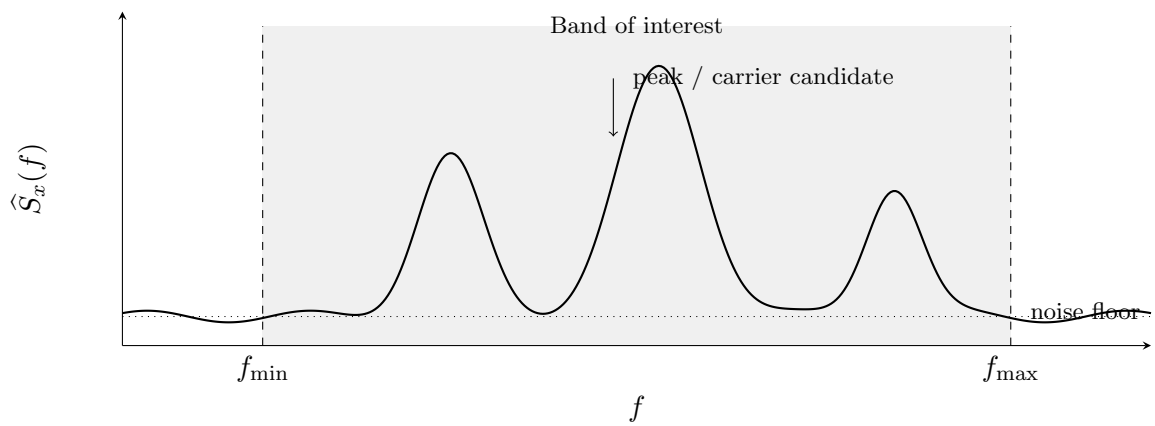


Figure 1: Illustration of a PSD estimate $\widehat{S}_x(f)$ of the received wideband complex I/Q over a frequency band of interest $[f_{\min}, f_{\max}]$, showing a noise floor and multiple detected emissions.

- *Center frequency (spectral centroid):*

$$f_{\text{cent}} \triangleq \frac{\int f\, \widehat{S}_x(f)\, df}{\int \widehat{S}_x(f)\, df},$$

  interpreted as a power-weighted mean frequency over the analyzed band.
- *SNR proxy:* estimate in-band signal power relative to a noise-floor estimate (median/robust fit outside detected carriers), producing $\widehat{\text{SNR}}$ for detection confidence and alert thresholds.

2. **Occupancy Detection & Band Discovery.** Convert $\widehat{S}_x(f)$ into a binary (or probabilistic) occupancy map by comparing against a noise-floor estimate $\widehat{N}(f)$:

$$\mathcal{O}(f) = \mathbf{1}\{\widehat{S}_x(f) > \widehat{N}(f) + \gamma\},$$

   where γ is a configurable margin (in dB). Connected occupied regions define candidate emissions; their centers, peak locations, and widths provide initial station candidates for downstream channelization and tracking.

3. **Quantitative Reconstruction of PSD.** When the PSD is degraded by noise, short observation windows, or coarse resolution, apply reconstruction to improve stability of the occupancy map and derived features. Examples include: (i) smoothing/averaging across time, (ii) interpolation across frequency bins, and (iii) model-based fitting for the noise floor and/or emission shapes. This stage must preserve auditability by logging parameters and reconstruction settings used for each reported estimate.

4. **Demodulation-Assisted Verification & Station Identification.** After candidate emissions are discovered from $\widehat{S}_x(f)$, each candidate channel is *isolated and decimated* (channelization $\rightarrow$ narrowband I/Q) and a lightweight FM demodulation stage is applied to:

   - *Confirm emission type:* verify that the detected occupancy corresponds to an FM broadcast-like signal (reject interferers and non-broadcast emissions).
   - *Extract identification cues:* detect the **19 kHz stereo pilot** and optionally decode **RDS** to obtain station metadata (e.g., PI/PS/PTY when available).

- *Support evidence generation:* produce short, time-stamped *audio snippets* and/or *RDS logs* linked to occupancy events for audits and investigations.
- *Enable demod-derived metrics (optional):* compute modulation/deviation-related indicators when required by the compliance rule set.

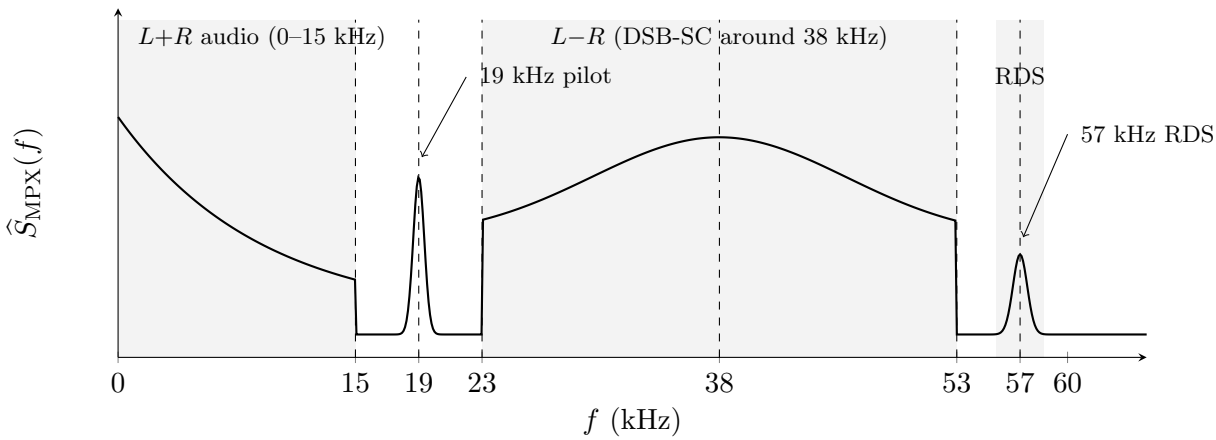


Figure 2: Demodulated baseband spectrum (post-demod) of the audio/composite (MPX) after FM discriminator, centered at 0 Hz audio, with features like 19 kHz pilot, 38 kHz stereo subcarrier, and possibly 57 kHz RDS.

Estimation of PSD $\widehat{S}_x(f)$ over a band of interest

```
import numpy as np
import SoapySDR
from SoapySDR import SOAPY_SDR_RX, SOAPY_SDR_CF32
from scipy.signal import firwin, lfilter, resample_poly
from fractions import Fraction
```

```
# ----------------------------
# User inputs (Hz)
# ----------------------------
f_min = 88e6
f_max = 108e6
f0    = 99.9e6          # station to demod (must be within [f_min, f_max])

# ----------------------------
# SDR capture configuration
# ----------------------------
bw_interest = f_max - f_min
f_center    = 0.5 * (f_min + f_max)

# HackRF commonly: 2..20 MS/s (choose >= interest bandwidth if you truly want
                                        the whole band)
Fs = max(2.0e6, 1.2 * bw_interest)   # add guard factor; clamp to your HW/USB
                                        limits
Fs = min(Fs, 20.0e6)

# Station offset (mix to baseband)
f_off = f0 - f_center
assert f_min <= f0 <= f_max, "f0 must be inside [f_min, f_max]"

# Narrowband FM channel settings
chan_bw = 200e3          # ~200 kHz for WBFM broadcast
audio_fs = 48_000
tau = 75e-6              # de-emphasis: 75 us (US); use 50e-6 in many other
                                        regions

# Choose an integer decimation to land near ~250 kS/s
target_fs = 250e3
D = max(1, int(round(Fs / target_fs)))
fs_nb = Fs / D

# FIR lowpass for channelization (cutoff slightly above channel BW/2)
numtaps = 129
cutoff = 0.6 * (chan_bw / 2.0)   # conservative
h = firwin(numtaps, cutoff/(Fs/2))

# Resample ratio to audio_fs
frac = Fraction(audio_fs / fs_nb).limit_denominator(512)
UP, DOWN = frac.numerator, frac.denominator
```

```
# ----------------------------
# SoapySDR HackRF setup
# ----------------------------
sdr = SoapySDR.Device(dict(driver="hackrf"))  # requires SoapyHackRF installed
sdr.setSampleRate(SOAPY_SDR_RX, 0, Fs)
sdr.setFrequency(SOAPY_SDR_RX, 0, f_center)
sdr.setGain(SOAPY_SDR_RX, 0, 20)             # adjust as needed (avoid overload)

rx = sdr.setupStream(SOAPY_SDR_RX, SOAPY_SDR_CF32, [0])
sdr.activateStream(rx)

# ----------------------------
# Streaming state
# ----------------------------
phase = 0.0
dphi = -2.0*np.pi * (f_off / Fs)              # negative => shift f0 to 0 Hz

zi_fir = np.zeros(len(h)-1, dtype=np.complex64)
last_x = 0j
deemph_state = 0.0

def deemphasis_iir(x, fs, tau, state):
# 1st-order lowpass: y[n] = a*y[n-1] + (1-a)*x[n]
a = np.exp(-1.0/(fs*tau))
y = np.empty_like(x, dtype=np.float32)
y0 = float(state)
for i, xi in enumerate(x):
y0 = a*y0 + (1.0-a)*float(xi)
y[i] = y0
return y, y0

# ----------------------------
# Main loop (CTRL+C to stop)
# ----------------------------
buff_len = 1 << 14
buff = np.empty(buff_len, np.complex64)

try:
while True:
sr = sdr.readStream(rx, [buff], buff_len, timeoutUs=200000)
if sr.ret <= 0:
continue

x = buff[:sr.ret]

# Mix station to baseband with phase continuity
n = np.arange(x.size, dtype=np.float32)
osc = np.exp(1j*(phase + dphi*n)).astype(np.complex64)
phase = (phase + dphi*x.size) % (2.0*np.pi)
x = x * osc

# Channel lowpass + decimate
x, zi_fir = lfilter(h, 1.0, x, zi=zi_fir)
x = x[::D]
```

```
# Quadrature FM demod (phase difference)
d = x * np.conj(np.r_[last_x, x[:-1]])
last_x = x[-1]
fm = np.angle(d).astype(np.float32)

# Resample to audio rate + de-emphasis
audio = resample_poly(fm, UP, DOWN).astype(np.float32)
audio, deemph_state = deemphasis_iir(audio, audio_fs, tau, deemph_state)

# At this point, ‘audio‘ is demodulated audio samples at audio_fs (float32)
# Example: compute RMS / store snippets / feed to encoder, etc.
rms = float(np.sqrt(np.mean(audio*audio) + 1e-12))
if rms > 0.05:
pass  # trigger recording / evidence logic

except KeyboardInterrupt:
pass
finally:
sdr.deactivateStream(rx)
sdr.closeStream(rx)
```

Note: n an SDR compliance monitor you have hard real-time ingress (HackRF USB host). If downstream processing or storage cant keep up, you get buffer overruns, latency blow-ups, or uncontrolled disk/network growth. A throttling/backpressure stage makes the system stable and predictable.